Interactive Geospatial Visualization Using Maps

Cheng Peng

West Chester University

There is a very rich set of tools for interactive geospatial visualization. This note introduces various R tools and Tableau to create interactive maps for visualizing spatial patterns.

Map Types

There are two common ways of representing spatial data on a map:

- Defining regions on a map and distinguishing them based on their value on some measure using colors and shading. This type of map is usually called `choropleth map`.
- Marking individual points on a map based on their longitude and latitude (e.g., archaeological dig sites; baseball stadiums; voting locations, etc.). This type of map is also called a `scatter map`.

Plotting `scatter maps` uses geocode and are relatively easier to create. However, a choropleth map is constructed using data with a special structure with shape information. It is relatively harder to construct a choropleth map.

A basemap provides context for additional layers that are overlaid on top of the basemap. Basemaps usually provide location references for features that do not change often like boundaries, rivers, lakes, roads, and highways. Even on basemaps, these different categories of information are in layers. Usually a basemap contains this basic data, and then extra layers with particular information from a particular data set, are overlaid on the base map layers for visual analysis.

In this note, the basemaps come primarily from the open-data-source-based OpenStreepMap.

Choropleth Map

Scatter Map

Leaflet Maps

We will use R `leaflet` library to create both reference maps and choropleth maps and plot data on maps to display spatial patterns.

Reference Maps

1. Introduction

`Leaflet` is one of the most popular open-source JavaScript libraries for interactive maps. It's used widely in practice. It has many nice features. R package `leaflet` allows us to make interactive maps using map tiles, markers, polygons, lines, and popups, etc.

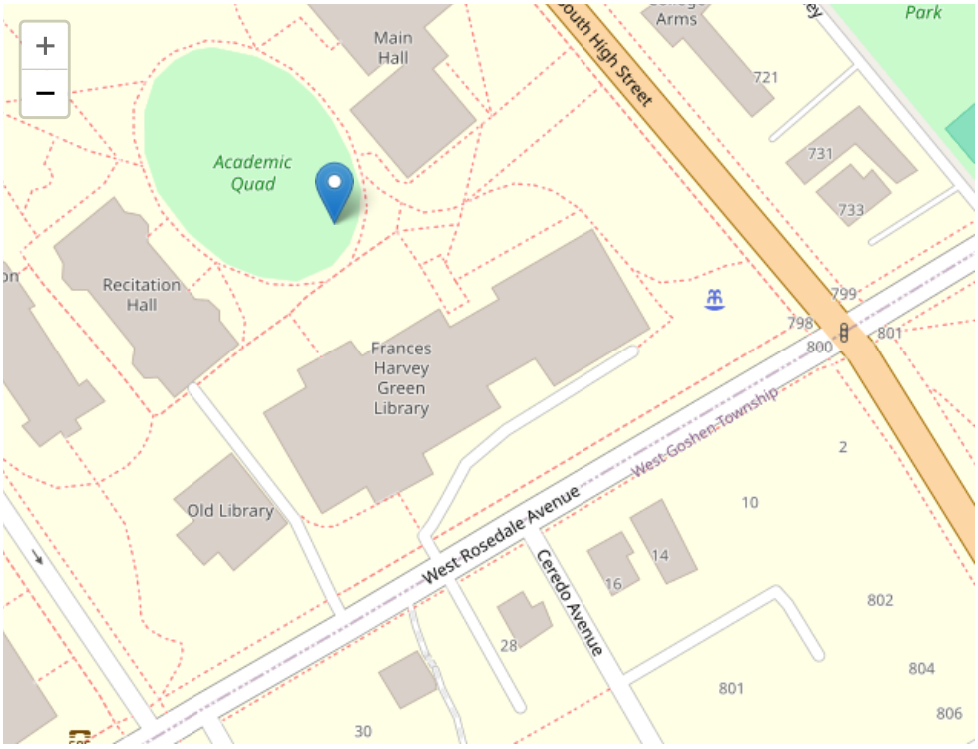
The function `leaflet()` returns a Leaflet map widget, which stores a list of objects that can be modified or updated later. Most functions in this package have an argument map as their first argument, which makes it easy to use the pipe operator `%>%`.

Creating a leaflet map with R library `leaflet` consisting of the following steps.

- Create a map widget by calling leaflet().
- Add layers (i.e., features) to the map by using layer functions (e.g. addTiles, addMarkers, addPolygons) to modify the map widget.
- Repeat the previous as desired.
- Print the map widget to display it.

Let's look at the following simple example.

```
# library(leaflet)        # it has been loaded in the setup chunk.
# define a leaflet map
m <- leaflet() %>%
     setView(lng=-75.5978, lat=39.9522, zoom = 20) %>%
     addTiles() %>%        # Add default OpenStreetMap map tiles
     addMarkers(lng=-75.5978, lat=39.9522)
m    # Print the map
```



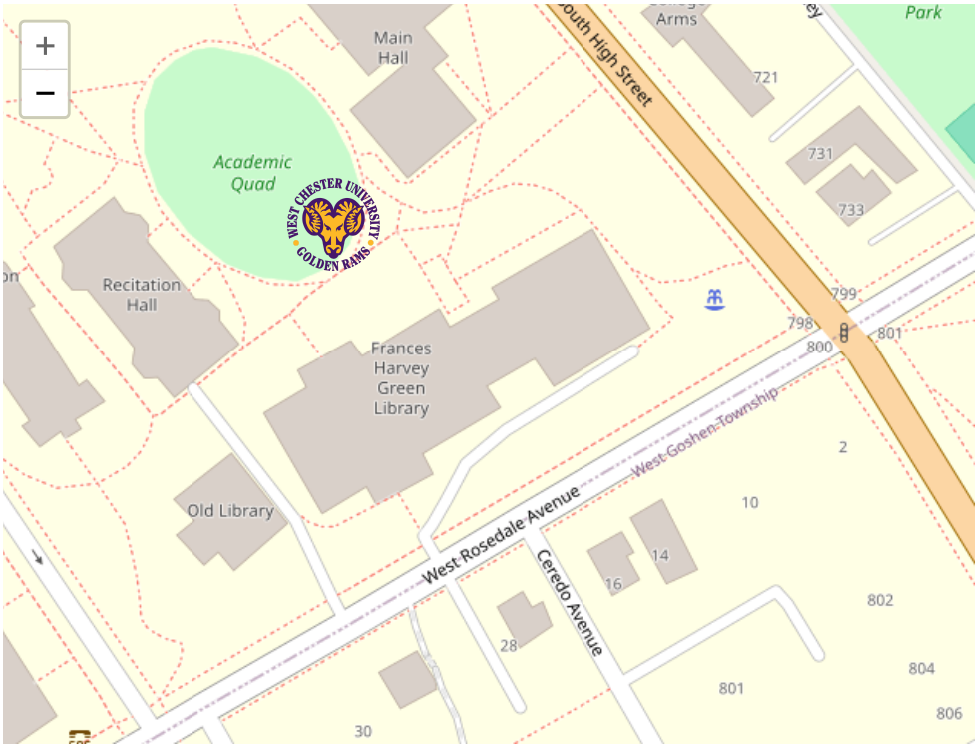
2. Customizing Marker Icons

We can manipulate the attributes of the map widget using a series of methods.

- `setView()` sets the center of the map view and the zoom level;
- `fitBounds()` fits the view into the rectangle [lng1, lat1] – [lng2, lat2];
- `clearBounds()` clears the bound, so that the view will be automatically determined by the range of latitude/longitude data in the map layers if provided;

We can also define our own markers and added to the map object. For example, we use WCU's logo as a custom marker and add it to the previous map.

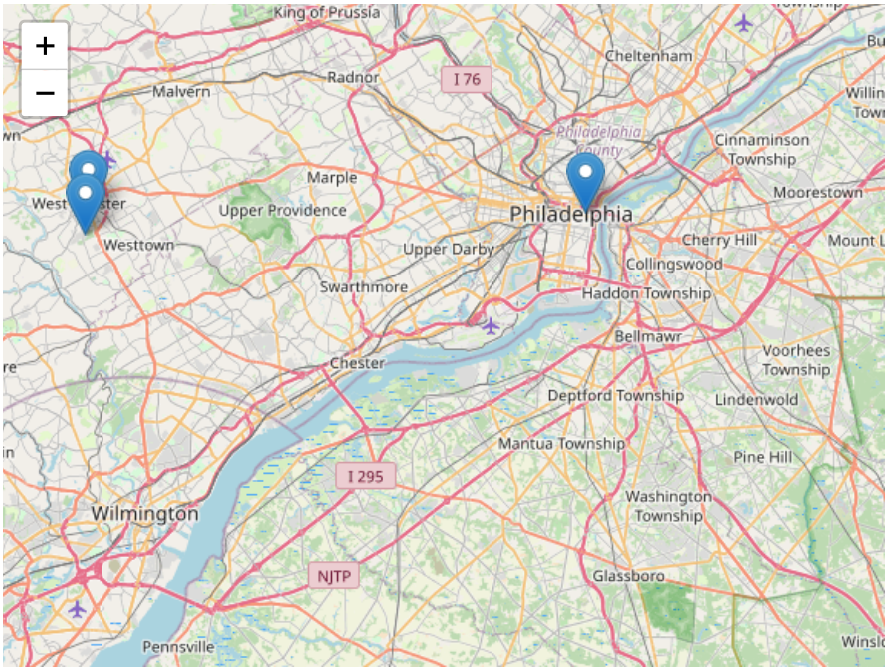
```
# define a marker using WCU's logo.
wcuicon <- makeIcon(
  iconUrl = "https://github.com/pengdsci/sta553/blob/main/image/goldenRamLogo.png?raw=true",
  iconWidth = 60, iconHeight = 60
  )
# define a leaflet map
m <- leaflet() %>%
     setView(lng=-75.5978, lat=39.9522, zoom = 20) %>%
     addTiles() %>%        # Add default OpenStreetMap map tiles
     addMarkers(lng=-75.5978, lat=39.9522,  icon = wcuicon)
m    # Print the map
```



3. Popups and Labels

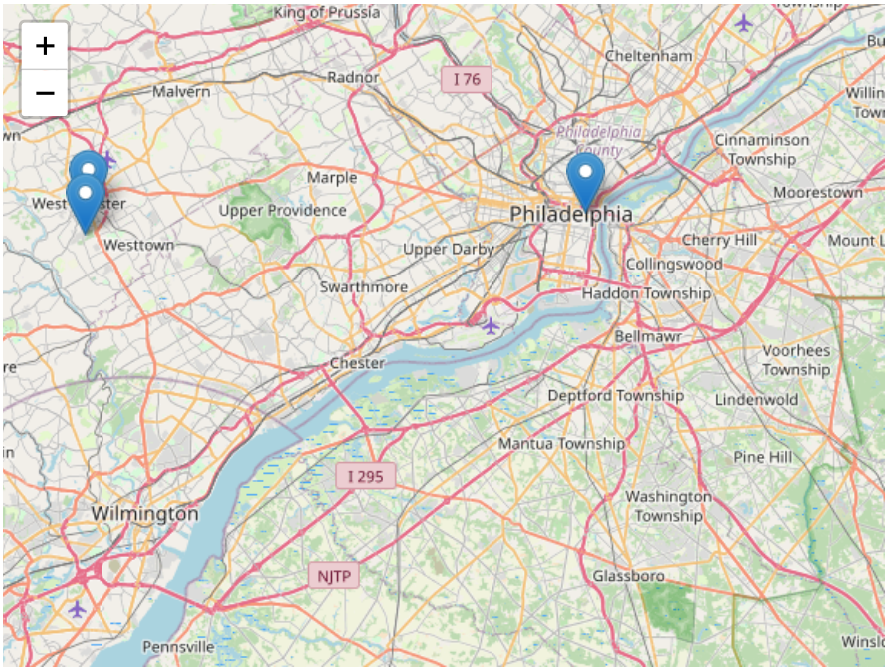
Popups are small boxes containing arbitrary HTML, that point to a specific point on the map. We can use the `addPopups()` function to add a standalone popup to the map. When you click the marker in the following map, you will see a popup with the name of the WCU campus.

```
df <- read.csv(textConnection(
"Name, Lat, Long
WCU Philadelphia Campus,39.9518,-75.1525
WCU South Campus,39.9373,-75.6011
WCU Main Campus, 39.9524,-75.5982"
))

leaflet(df) %>%
  addTiles() %>%
  setView(lng=-75.3768, lat=39.9448, zoom = 10) %>%
  addMarkers(~Long, ~Lat, popup = ~paste("Name: ",Name))
```



We can also change the popups in the above map to labels. The modified code is shown below

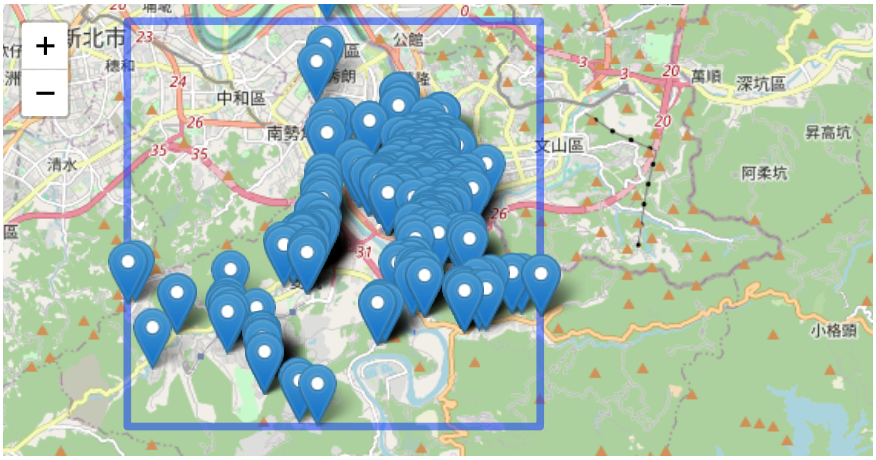
```r
df <- read.csv(textConnection(
"Name, Lat, Long
WCU  Philadelphia Campus,39.9518,-75.1525
WCU  South Campus,39.9373,-75.6011
WCU  Main Campus, 39.9524,-75.5982"
))

leaflet(df) %>%
  addTiles() %>%
  setView(lng=-75.3768, lat=39.9448, zoom = 10) %>%
  addMarkers(~Long, ~Lat, label = ~paste("Name:", Name))
```



4. Examples Using Real-World Data

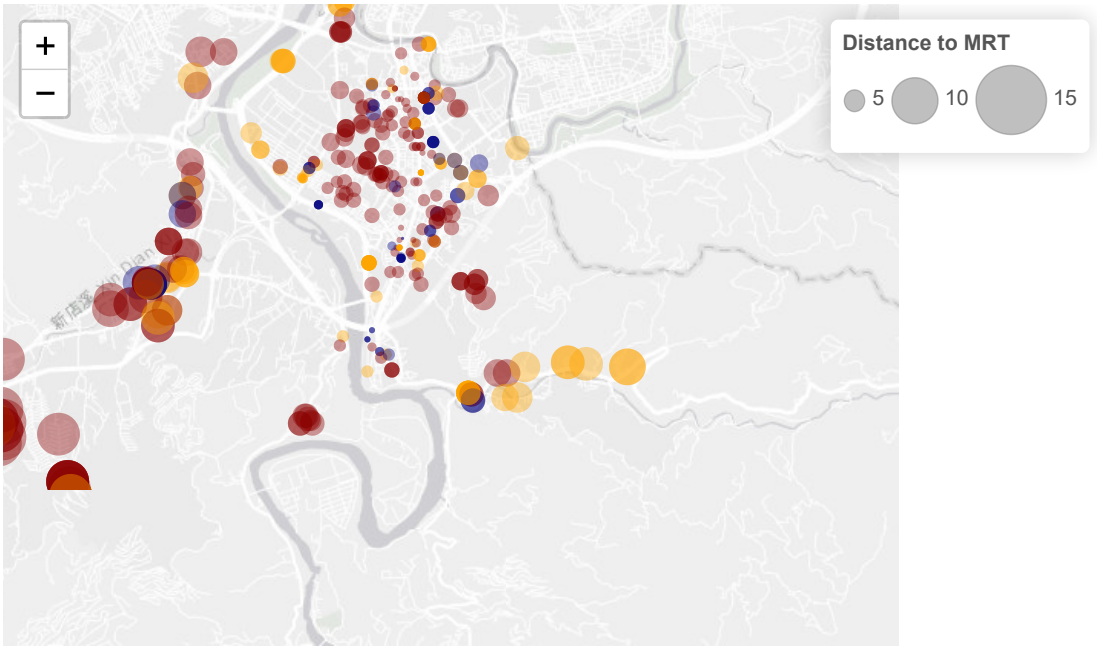
In the following example, we use a few leaflet functions to add some features such as drawing highlight boxes, labels, etc. to the map.

```
# Define bounding box using the range of longitude/latitude coordinates
# from the given data set
housing.price <- read.csv("Realestate.csv")
# making static leaflet map
leaflet(housing.price) %>%
  addTiles() %>%
  setView(lng=mean(housing.price$Longitude), lat=mean(housing.price$Latitude), zoom = 14) %>%
   addRectangles(
    lng1 = min(housing.price$Longitude), lat1 = min(housing.price$Latitude),
    lng2 = max(housing.price$Longitude), lat2 = max(housing.price$Latitude),
    #fillOpacity = 0.2,
    fillColor = "transparent"
    ) %>%
  fitBounds(
    lng1 = min(housing.price$Longitude), lat1 = min(housing.price$Latitude),
    lng2 = max(housing.price$Longitude), lat2 = max(housing.price$Latitude) ) %>%
  addMarkers(~Longitude, ~Latitude, label = ~PriceUnitArea)
```



In the next map based on the data, we add more information to that map to display higher dimensional information.

```r
housing.price <- na.omit(read.csv("Realestate.csv"))
## color coding a continuous variable:
colAge <- cut(housing.price$HouseAge, breaks=c(0, 5, 15, max(housing.price$HouseAge)+1), right = FALSE)
colAgeNum <- as.numeric(colAge)
##
ageColor <- rep("navy", length(colAge))
ageColor[which(colAgeNum==2)] <- "orange"
ageColor[which(colAgeNum==3)] <- "darkred"
## define label with hover messages

label.msg <- paste(paste("Unit Price:", housing.price$PriceUnitArea),
                   paste("\n Dist to MRT:",housing.price$Distance2MRT),"\n")

#labels = cat(label.msg)
# making leaflet map
leaflet(housing.price) %>%
  addTiles() %>%
  setView(lng=mean(housing.price$Longitude), lat=mean(housing.price$Latitude), zoom = 13) %>%
  #OpenStreetMap, Stamen, Esri and OpenWeatherMap.
  addProviderTiles("Esri.WorldGrayCanvas") %>%
  addCircleMarkers(
             ~Longitude,
             ~Latitude,
             color = ageColor,
             radius = ~ sqrt(housing.price$Distance2MRT/10)*0.7,
             stroke = FALSE,
             fillOpacity = 0.4,
             label = ~label.msg)  %>%
  addLegend(position = "bottomright",
             colors = c("navy", "orange","darkred"),
             labels= c("[0,5)", "[5,15)", "[15,44.8)"),
             title= "House Age",
             opacity = 0.4) %>%
  addLegendSize(position = 'topright',
                  values = sqrt(housing.price$Distance2MRT/10)*0.7,
                   color = 'gray',
                fillColor = 'gray',
                  opacity = .5,
                   title = 'Distance to MRT',
                   shape = 'circle',
              orientation = 'horizontal',
                   breaks = 5)
```


+
−
Distance to MRT
5
10
15
House Age
[0,5)
[5,15)
[15,44.8)
Leaflet | © OpenStreetMap, ODbL, Tiles © Esri — Esri, DeLorme, NAVTEQ

Plotly Map

`plotly` aims to be a general-purpose visualization library, and thus, doesn't aim to be the most fully-featured geospatial visualization toolkit.

`plotly` uses several different ways to create maps – each with its strengths and weaknesses. It utilizes plotly.js's built-in support to render the basemap layer. The types of basemap used in plotly are Mapbox (third party software that requires an access token) and D3.js powered basemap. In other words, plotly does not use OpenStreetMap that is used in `leaflet`, `Mapviewer`, `ggplot2`,`Shiny`, and Tableau.

We will not use `Mapbox` in this note and focus on the D3.js basemap that does not have many details. The plot function `plot_geo()` will be used to make quick maps.

Choropleth Maps

In the following, we will introduce the steps for creating Choropleth maps. Since Choropleth maps need to fill and color small regions such as district, county, states, etc., it requires the data set to have a special structure that contains shape information. Two plot constructor functions `plot_ly()` and `plot_geo()` will be introduced to create choropleth maps.

- `plot_ly()` requires specifying `type = choropleth` to make a map (basemap from plotly.js). Information in the data set is integrated to the maps by various arguments of `plot_ly()` and relevant graphic functions that are compatible with `plot_ly()`.
- `plot_geo()` requires `addTrace()` to make choropleth maps and integrate data information to the maps with relevant arguments in `addTrace()` and graphic functions compatible with `plot_geo()`.

1. Choropleth Maps with `plot_ly()`

In general, making choropleth maps with `plot_ly()` requires two main types of input:

- Geometry information provided by
  - one of the built-in geometries within plot_ly such as US states and world countries. See the following example 1: visualizing 2018 electricity cost per state.
  - a supplied GeoJSON file where each feature has either an id field or some identifying value in properties. See the following example 2: visualizing the unemployment rate of US counties
- A list of values indexed by feature identifier. They control the features in the map including boundary, filled colors, legend, hover text, etc.

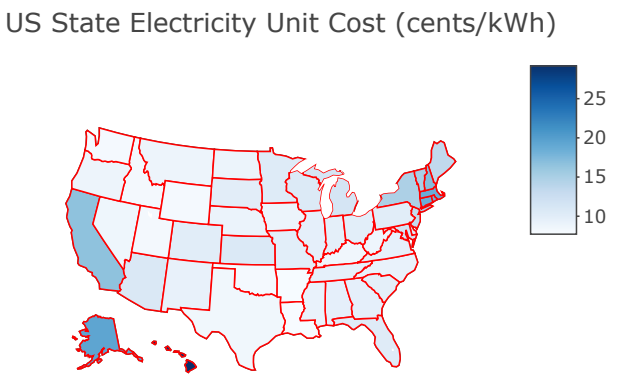
For the US map, two types of projections were commonly use: regulary and Albers.

Example 1: US electricity cost by States in 2018. The arguments `locations =` and `locationmode =` tell `plot_ly` what map information should be used to create the base map. Other arguments and functions are used to control different features of the resulting map. One cautionary note is that `plot_ly()` only uses state abbreviations as the state name.

In the code, the state abbreviations `state.abb` is a built-in data set. Several other built-in data sets about each state are also available. Check the website http://stats4stem.weebly.com/r-statex77-data.html for more information on these data sets.

```
# Map data preparation
electricitycost <-read.csv("https://github.com/pengdsci/sta553/raw/main/data/state_electricity_data_2018
electricitycost$State <- state.abb  # add state abbrevs to specify locations in plot_ly()
# Create hover text
electricitycost$hover <- with(electricitycost, paste(State, '<br>', "Electricity Cost:", centskWh))
# Make state borders white
borders <- list(color = toRGB("red"))
# Set up some mapping options
map_options <- list(
  scope = 'usa',
  projection = list(type = 'albers usa'),
```

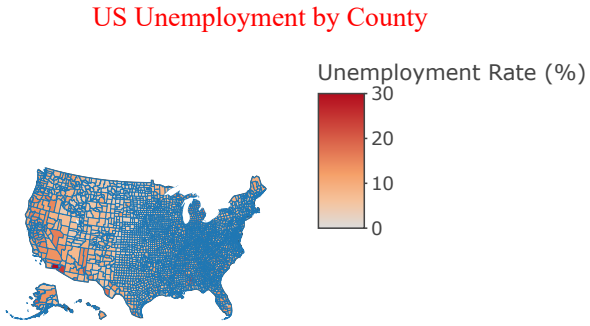
```
  showlakes = TRUE,
  lakecolor = toRGB('white')
)
plot_ly( z = ~electricitycost$centskWh,
        text = ~electricitycost$hover,
        locations = ~electricitycost$State,
        type = 'choropleth',
        locationmode = 'USA-states',
        color = electricitycost$centskWh,
        colors = 'Blues',
        marker = list(line = borders)) %>%
  layout(title = 'US State Electricity Unit Cost (cents/kWh)',
         geo = map_options)
```



Example 2: The unemployment rates of US counties. This example requires a JSON file to provide necessary geometric information (shape polygon) about each county in the US. The argument `locations =` accepts FIPS (Federal Information Process System) for US county maps. The geometric information of the US county

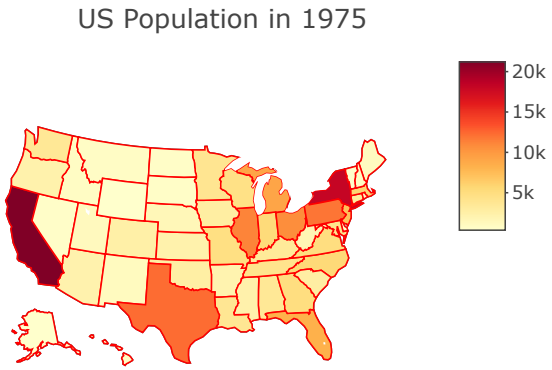
shape is supplied in a JSON file and used through the argument `geojson =`.

```
#library(plotly)
#library(rjson)
#library("RColorBrewer")  # brewer.pal.info for list of color scales

url <- 'https://github.com/pengdsci/sta553/raw/main/data/geojson-counties-fips.json'  # contains geocod
counties <- rjson::fromJSON(file=url)
load("img07//unemp.rda")
#load("/Users/chengpeng/WCU/Teaching/2022Spring/STA553/RMaps/unemp.rda")
df=unemp
g <- list(
      scope = 'usa',
      projection = list(type = 'albers usa'),
      showlakes = TRUE,
      lakecolor = toRGB('white')
    )
###
fig <- plot_ly()  %>%
  add_trace( type = "choropleth",
          geojson = counties,
        locations = df$fips,
                z = df$rate,
       colorscale = "GnBu",
             zmin = 0,
             zmax = 30,
             text = df$name, # hover mesg
           marker = list(line=list(width=0.2))
          )   %>%
  colorbar(title = "Unemployment Rate (%)",
           colorscale='Viridis')  %>%
  layout( ### Title
          title =list(text = "US Unemployment by County",
                                font = list(family = "Times New Roman",  # HTML font family
                                            size = 18,
                                           color = "red")),
          geo = g)
fig
```



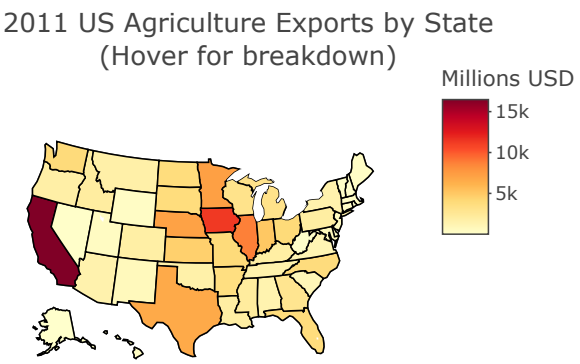
Example 3 US states facts. Similar to example 1, but with more variables. The data set is built-in in the base R package.

```r
# Create data frame
state_pop <- read.csv("https://raw.githubusercontent.com/pengdsci/sta553/main/data/USStatesFacts.csv")
# Create hover text
state_pop$hover <- with(state_pop,
                        paste(STName, '<br>', "Population:", Population,
                              '<br>', "Income:", Income,
                              '<br>', "Life.Exp:", Life.Exp,
                              '<br>', "Murder:", Murder,
                              '<br>', "HS.Grad:", HS.Grad))
# Make state borders white
borders <- list(color = toRGB("red"))
# Set up some mapping options
map_options <- list(
  scope = 'usa',
  projection = list(type = 'regular usa'),
  showlakes = TRUE,
  lakecolor = toRGB('white')
)
plot_ly(z = ~state_pop$Population,
        text = ~state_pop$hover,
        locations = ~state_pop$State,
        type = 'choropleth',
        locationmode = 'USA-states',
        color = state_pop$Population,
        colors = 'YlOrRd',
        marker = list(line = borders)) %>%
  layout(title = 'US Population in 1975', geo = map_options)
```



2. Choropleth Maps with `plot_geo()` Making a choropleth map with `plot_geo` requires less effort to prepare the shape data. The geo-information was called through `locations =` and `locationmode =`.

```r
# library(plotly)
# read in cv data
df <- read.csv("https://raw.githubusercontent.com/pengdsci/sta553/main/data/2011_us_ag_exports.csv")
## Define hover text
df$hover <- with(df, paste(state, "<br>",
                           "Beef", beef, "<br>",
                           "Dairy", dairy, "<br>",
                           "Fruits", total.fruits, "<br>",
                           "Veggies", total.veggies, "<br>",
                           "Wheat", wheat, "<br>",
                           "Corn", corn))
# give state boundaries a white border
l <- list(color = toRGB("white"), width = 2)
# specify some map projection/options
g <- list(     scope = 'usa',
          projection = list(type = 'albers usa'),
           showlakes = TRUE,
           lakecolor = toRGB('white')
      )
## plot map
m <- plot_geo(df, locationmode = 'USA-states') %>%
     add_trace(        z = ~total.exports,
                    text = ~hover,
               locations = ~code,
                   color = ~total.exports,
                  colors = 'YlOrRd'
              )  %>%
     colorbar(title = "Millions USD")  %>%
     layout( title = '2011 US Agriculture Exports by State<br>(Hover for breakdown)',
               geo = g
           )
m
```



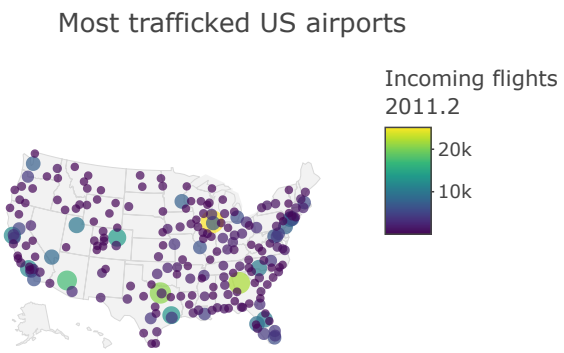
Scatter Maps

3.Scatter Map with `plot_geo()` and `add_markers()`

A Scatter map is relatively easier to make since we only plot the base map using the longitude and latitude. No map shape information is needed for scatter maps.

Example 4 US Airport Traffic.

```
#library(plotly)
df <- read.csv('https://raw.githubusercontent.com/pengdsci/sta553/main/data/2011_february_us_airport_tra
# geo styling
g <- list(      scope = 'usa',
           projection = list(type = 'albers usa'),
             showland = TRUE,
            landcolor = toRGB("gray95"),
         subunitcolor = toRGB("gray85"),
         countrycolor = toRGB("gray85"),
         countrywidth = 0.5,
         subunitwidth = 0.5
       )
###
fig <- plot_geo(df, lat = ~lat, lon = ~long) %>%
  add_markers( text = ~paste(airport, city, state,
                             paste("Arrivals:", cnt),
                             sep = "<br>"),
               color = ~cnt,
               symbol = "circle",
               size = ~cnt,
               hoverinfo = "text")   %>%
  colorbar(title = "Incoming flights<br>2011.2")  %>%
  layout( title = 'Most trafficked US airports',
          geo = g )

fig
```


Most trafficked US airports
Incoming flights
2011.2
20k
10k

Custom Maps

4. Custom Map With Special Libraries

Sometimes, we may want to use custom maps to represent spatial information. For example, if we want to visualize the area of US states, the previous US maps are fine. If we want to represent the population size (Example 3), we may want to use a map such that the displayed area is proportional to the population size but not the geographical area. These types of custom maps need special tools to construct. Show you an exam without providing code to make the map.

Example 4 US population by states.

Thematic Maps

The `tmap` package is a relatively new way to plot thematic maps in R. Thematic maps are geographical maps in which spatial data distributions are visualized. This package offers a flexible and layer-based approach to creating thematic maps, such as choropleths and bubble maps. The syntax for creating plots is similar to that of ggplot2.

`tmap_mode()` will be used to determine interactivity of the map: `tmap_mode("plot")` produce static maps and `tmap_mode("view")` produce interactive maps.

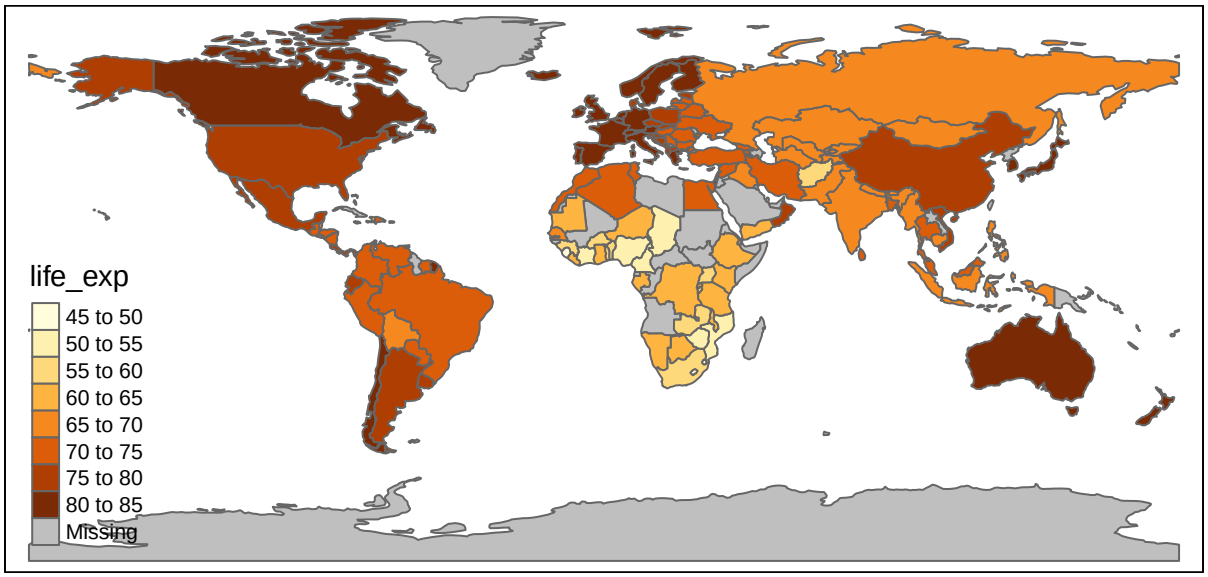
1. Choropleth Maps

We will use a built-in world shapefile, `World`, that contains information about population, gdp, life expectancy, income, happiness index, etc.

Both choropleths and scatter maps will be illustrated using the built-in data.

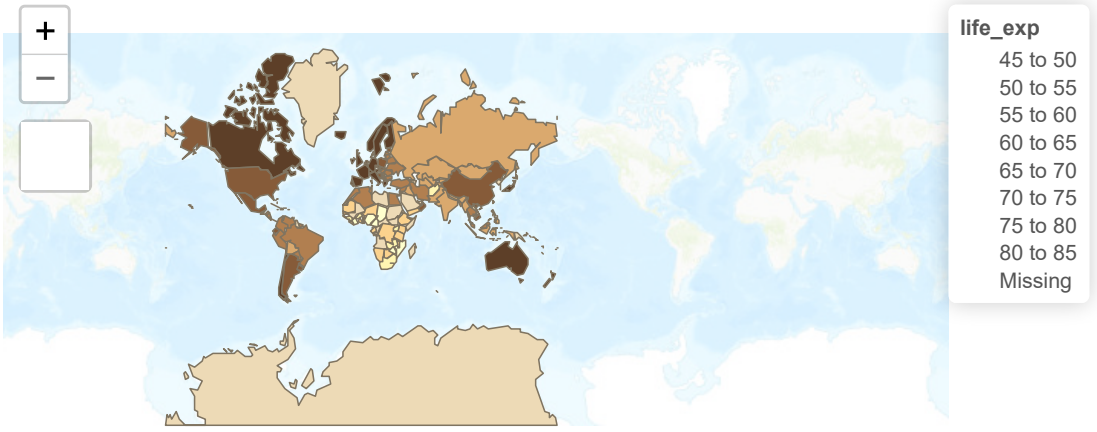
Example 1: Choropleths: The default world map with the distribution of mean life expectancy among all countries. The default `tmap_mode` is set to be `plot`. The default `tmap` map is static.

```
library(tmap)
data(World)
tm_shape(World) +
    tm_polygons("life_exp")
```



Example 2: Interactive Map: use the above static map as the base map and add interactive features to the maps. The mode can be set with the function `tmap_mode()`, and toggling between the modes can be done with the 'switch' `ttm()` (which stands for toggle thematic map.

```
library(tmap)
#
tmap_mode("view")  # "view" gives interactive map; "plot" gives static map.
##
## tmap_style set to "classic"
tmap_style("classic")
## other available styles are: "white", "gray", "natural",
## "cobalt", "col_blind", "albatross", "beaver", "bw", "watercolor"
tmap_options(bg.color = "skyblue",
             legend.text.color = "white")
##
tm_shape(World) +
      tm_polygons("life_exp",
                  legend.title = "Life Expectancy") +
      tm_layout(bg.color = "gray",
                inner.margins = c(0, .02, .02, .02))
```

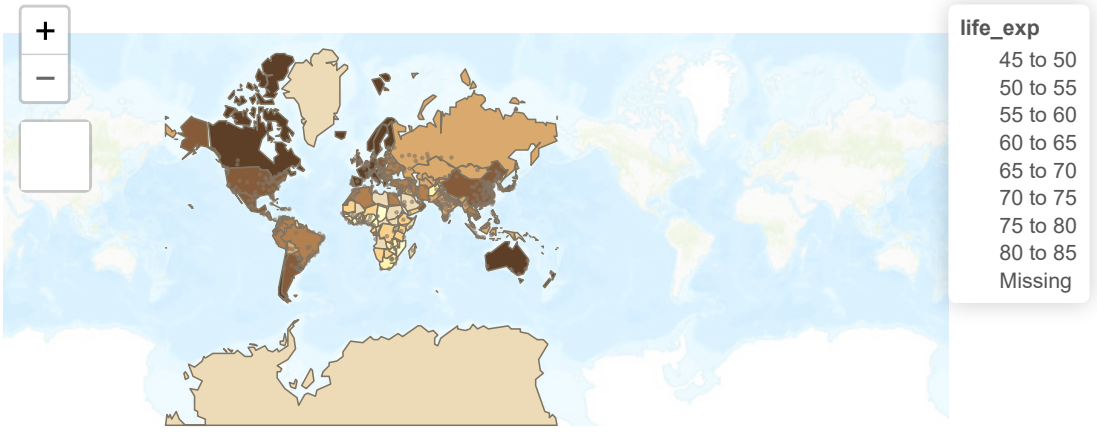


Mixed Maps (Multilayer)

Through a multilayer map, we can make a choropleth and place another on top of it. In the following example, we add one additional layer using the `metro` shapefile and plot the center of the metro area to get a scatter map.

Example 3: Mixed Map: two-layer mixes maps

```
library(tmap)
#*
data(metro)
##
tmap_mode("view")  # "view" gives interactive map;
#tmap_style("classic") ## tmap_style set to "classic"
## other available styles are: "white", "gray", "natural",
## "cobalt", "col_blind", "albatross", "beaver", "bw", "watercolor"
tmap_options(bg.color = "skyblue",
             legend.text.color = "white")
##
tm_shape(World) +
      tm_polygons("life_exp",
                  legend.title = "Life Expectancy") +
      tm_layout(bg.color = "gray",
                inner.margins = c(0, .02, .02, .02)) +
tm_shape(metro) +
      tm_symbols(col = "purple",
                 size = "pop2020",
                 scale = .5,
                 alpha = 0.5,
                 popup.vars=c("pop1950", "pop1960", "pop1980","pop1990",                    "pop2000","p
```

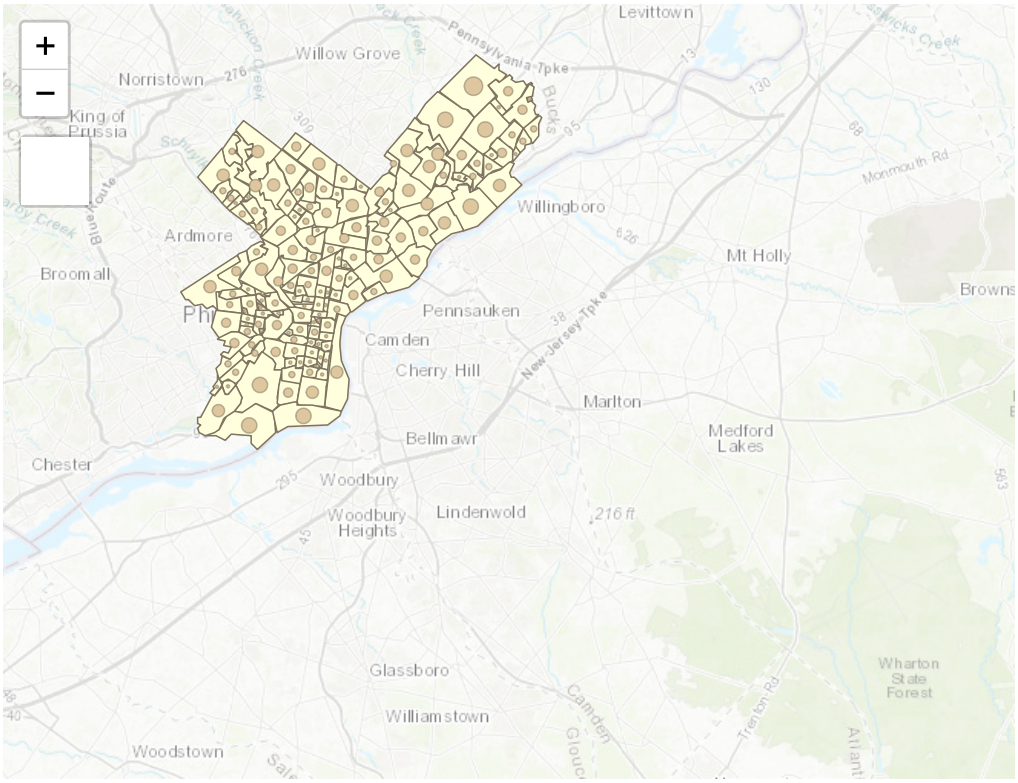



```
library(spData)
library(sf)
```

```
library(mapview)
gj = "https://github.com/azavea/geo-data/raw/master/Neighborhoods_Philadelphia/Neighborhoods_Philadelphi
##
gjsf = st_read(gj)
```

```
Reading layer `Neighborhoods_Philadelphia' from data source
  `https://github.com/azavea/geo-data/raw/master/Neighborhoods_Philadelphia/Neighborhoods_Philadelphia.g
  using driver `GeoJSON'
Simple feature collection with 158 features and 8 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -75.28027 ymin: 39.867 xmax: -74.95576 ymax: 40.13799
Geodetic CRS:  WGS 84
```

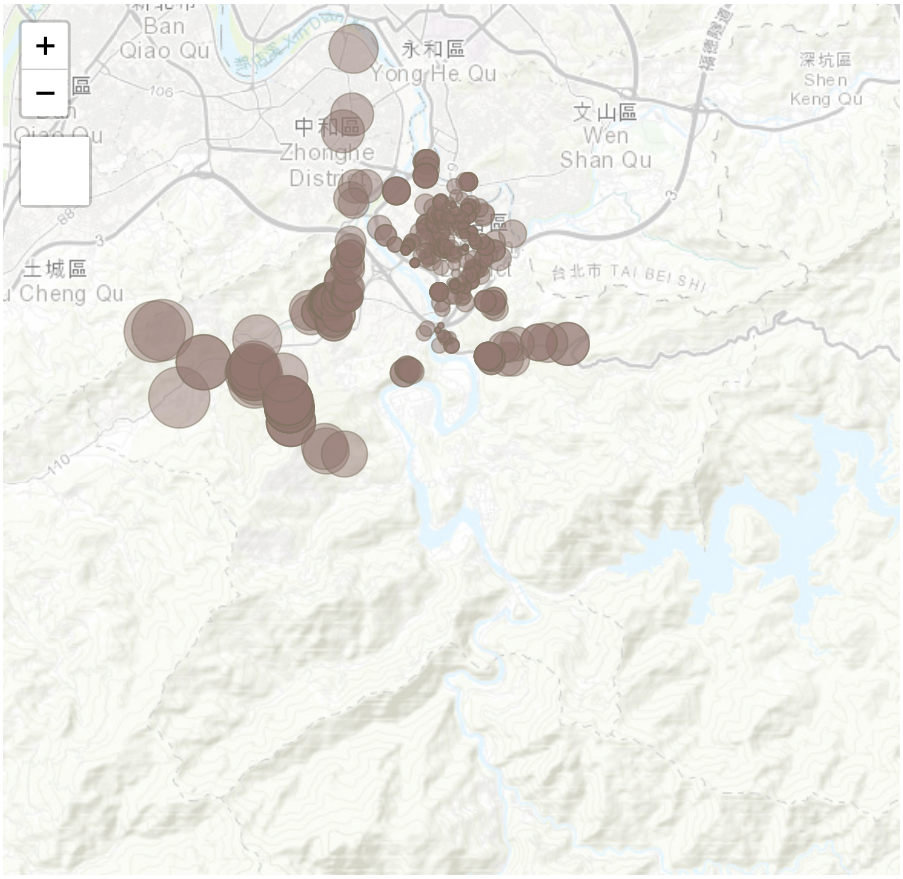
```
library(tmap)
#    tm_shape(World) +
    tm_shape(gjsf)  +
      tm_polygons(legend.show = FALSE)  +
      tm_bubbles("shape_area",
                 #col = "shape_area",
                 #breaks=seq(1276674, 129254597, length = 6),
                 palette="-RdYlBu",
                 contrast=1)
```



Scatter Maps With geoCoordinates

We use the real estate data set to make a scatter map using library `tmap`. We first need to define an `sf` object using `st_as_sf()` that shapefile with an individual point based on the longitude and latitude in the data set.

```
library(tmap)
library(sf)
realestate0 <- read.csv("Realestate.csv", header = TRUE)
realest <- realestate0[, -1]
## create a shapefile with POINT type.
realest <-st_as_sf(realest, coords=c("Longitude","Latitude"), crs = 4326)
###
tm_shape(realest) +
  tm_dots(col = "purple",
          size = "Distance2MRT",
          alpha = 0.5,
          popup.vars=c("HouseAge", "PriceUnitArea", "NumConvenStores"),
          shapes = c(1, 0))
```



Leaflet | Tiles © Esri — Esri, DeLorme, NAVTEQ, TomTom, Intermap, iPC, USGS, FAO, NPS, NRCAN, GeoBase, Kadaster NL, Ordnance Survey, Esri Japan, METI, Esri China (Hong Kong), and the GIS User Community

Tableau Maps

We create a choropleth map and a scatter map respectively in this note. Before creating maps, we first look understand the structure of Tableau's sheet.

We can see that each Tableau book has several components:

- **A - Workbook name** - A workbook contains sheets. A sheet can be a worksheet, a dashboard, or a story.
- **B - Cards and shelves** - Drag fields to the cards and shelves in the work-space to add data to your view.
- **C - Toolbar** - Use the toolbar to access commands and analysis and navigation tools.
- **D - View** - This is the canvas in the work-space where you create a visualization (also referred to as a "viz").
- **E - Start page icon** - Click this icon to go to the Start page, where you can connect to data. For more information, see Start Page.
- **F - Side Bar** - In a worksheet, the side bar area contains two tabs: the Data pane and the Analytics pane.
- **G - Data Source** - Click this tab to go to the Data Source page and view your data.
- **H - Status bar** - Displays information about the current view.
- **I - Sheet tabs** - Tabs represent each sheet in your workbook. This can include worksheets, dashboards, and stories. You can rename and add more of these sheets, dashboards and stories if needed.
- **Show Me** - Click this toggle to select 24 built-in charts and the information needed to create these charts.

Choropleth Map

The data set we use for a choropleth map can be downloaded from https://raw.githubusercontent.com/pengdsci/sta553/main/data/USStatesFacts.csv.

You need to download save this data file in a folder and then connect it to Tableau Public (or Tableau Online).

The following are steps for making a choropleth map:

1. Load the .csv file to Tableau (Public);
2. Click `sheet1` in the bottom left taskbar;
3. Drag variable `State` (on the left navigation panel under the table) to the main drop field (Tableau considers `State` as a geo-variable); at the same time, the two generated `Longitude(generated)` and `Latitude(generated)` appear in the column and row fields automatically.
4. Click the `Show Me` (on the right side of a tiny color bar chart) in the top right of the screen;
5. You will see a list of graphs. Click the middle `world map` in the second row, you will see an initial choropleth map.
6. Click `Show Me` again to close the popup. We can click the legend on the top-right color to change the color of the map (if you like).
7. To add more information to the hover text, you drag the variables on the list to the small icon labeled with `Detail`.
8. Click `Sheet 1` to change it to a meaningful title.

9. Finally we label the states by their abbreviations. To do this, drag `State` to `Label` in the `Marks` table (next to `Detail`).
10. You can edit the hover text by clicking `Tooltip`.

The resulting map can be viewed on the Tableau Public Server at https://public.tableau.com/app/profile/cpeng/viz/US-States-Facts/Sheet1

Scatter Map

We use housing price data with longitude and latitude associated with each property. The data set is at https://projectdat.s3.amazonaws.com/Realestate.csv

As we did in the previous example, we download the data set and save it in a folder.

The following are steps to create a scatter map.

1. Open the Tableau and connect the data source to Tableau.
2. After the data has been loaded to the Tableau, click `Sheet1`, you will see the list of variables on the left panel.
3. click `Latitude` -> `Geographic Role` -> `Latitude`; do the same thing to `Longitude`.
4. Drag `Latitude` to the `Columns` field and `Longitude` to the `Rows` field. You will see a single point in `Sheet 1`. The two variables were automatically renamed as `AVG(Latitude)` and `AVG(Longitude)`.
5. Click `AVG(Latitude)` and select `dimension`, you will see a line plot in `Sheet 1`. Do the same thing to `AVG(Longitude)`. Now you see a scatter plot.
6. Click `Show Me` (top-right corner of `Book 1`) and select the left-hand side map icon (the first one in the second row), you will see an initial scatter map.
7. We want to use the size of the point to reflect the unit price. we drag `PriceUnitArea` to `Size` card in the `Marks` shelf.
8. Click `Show Me` to close the chart menu. Click `SUM(Price Unit Area)` (top-right corner) to change the point size.
9. I drag `Transaction Year` to the `Color` card to reflect the transaction year. We should choose a divergent color scale.
10. drag variables to the `Detail` card to be shown in the hover text.
11. Since many unit prices are close to each other, there are overlapped points. So we want to change the level of opacity. To do this, click `Color` card, choose the appropriate level of opacity, and edit the color to make a better map.
12. Add a meaningful title.
13. Right click the map and select `Map Layers` make the changes on the map background and layers.
14. Other edits and modifications to improve map.

The resulting map can be viewed on the Tableau Public Server at https://public.tableau.com/app/profile/cpeng/viz/RealEstateData_16469067466610/Sheet1

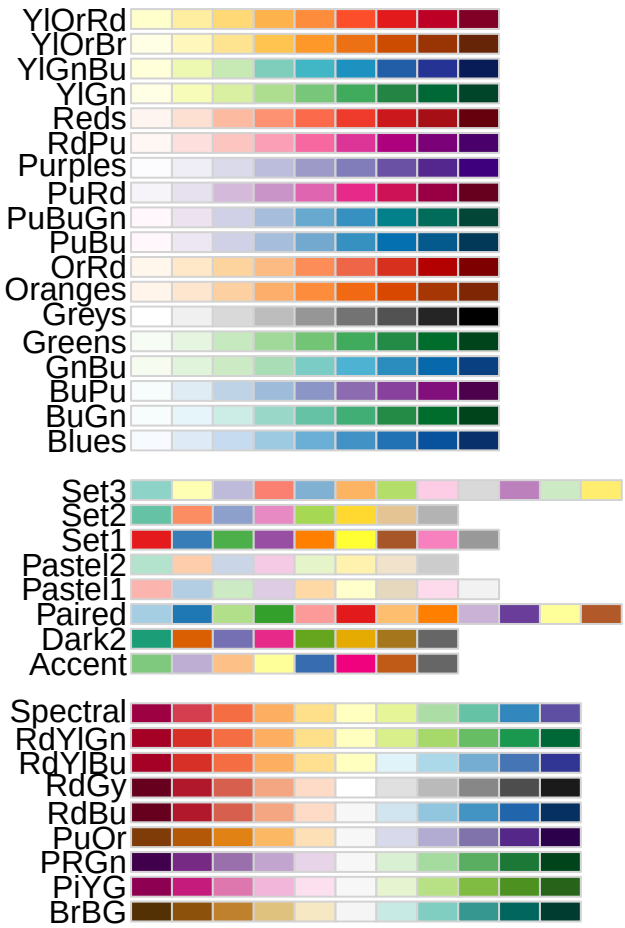
R Color Palettes

Since color coding is particularly important in map representation. We can use the following code to view various defined color scales (continuous and discrete) in the R library `RColorBrewer`. * **Sequential palettes** are suited to ordered data that progress from low to high (gradient). The palettes names are : `Blues`, `BuGn`, `BuPu`, `GnBu`, `Greens`, `Greys`, `Oranges`, `OrRd`, `PuBu`, `PuBuGn`, `PuRd`, `Purples`, `RdPu`, `Reds`, `YlGn`, `YlGnBu`, `YlOrBr`, `YlOrRd`.

- **Qualitative palettes** are best suited to represent nominal or categorical data. They do not imply magnitude differences between groups. The palettes names are : `Accent`, `Dark2`, `Paired`, `Pastel1`, `Pastel2`, `Set1`, `Set2`, `Set3`.
- **Diverging palettes** put equal emphasis on mid-range critical values and extremes at both ends of the data range. The diverging palettes are : `BrBG`, `PiYG`, `PRGn`, `PuOr`, `RdBu`, `RdGy`, `RdYlBu`, `RdYlGn`, `Spectral`.

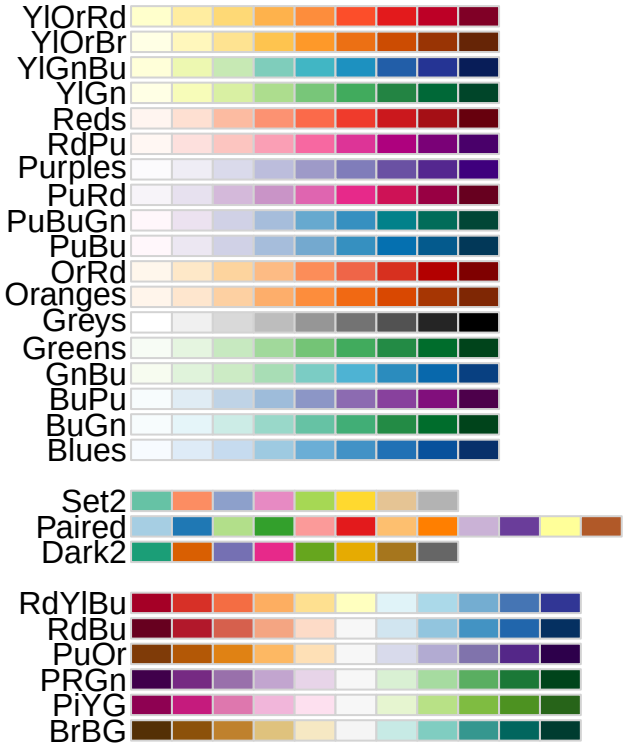
All Palettes

```
library("RColorBrewer")
display.brewer.all()
```



2. Color-bind Friendly Palettes

```
library("RColorBrewer")
display.brewer.all(colorblindFriendly = TRUE)
```



3. Color Palette Codes

```
library("RColorBrewer")
kable(brewer.pal.info)
```

	maxcolors	category	colorblind
BrBG	11	div	TRUE
PiYG	11	div	TRUE
PRGn	11	div	TRUE
PuOr	11	div	TRUE
RdBu	11	div	TRUE
RdGy	11	div	FALSE
RdYlBu	11	div	TRUE
RdYlGn	11	div	FALSE
Spectral	11	div	FALSE
Accent	8	qual	FALSE

	maxcolors	category	colorblind
Dark2	8	qual	TRUE
Paired	12	qual	TRUE
Pastel1	9	qual	FALSE
Pastel2	8	qual	FALSE
Set1	9	qual	FALSE
Set2	8	qual	TRUE
Set3	12	qual	FALSE
Blues	9	seq	TRUE
BuGn	9	seq	TRUE
BuPu	9	seq	TRUE
GnBu	9	seq	TRUE
Greens	9	seq	TRUE
Greys	9	seq	TRUE
Oranges	9	seq	TRUE
OrRd	9	seq	TRUE
PuBu	9	seq	TRUE
PuBuGn	9	seq	TRUE
PuRd	9	seq	TRUE
Purples	9	seq	TRUE
RdPu	9	seq	TRUE
Reds	9	seq	TRUE
YlGn	9	seq	TRUE
YlGnBu	9	seq	TRUE
YlOrBr	9	seq	TRUE
YlOrRd	9	seq	TRUE

4. Functions for Selecting Specific Color Palettes

Two functions can be used to display a specific color palette or return the code of the palette.

- `display.brewer.pal(n, name)` displays a single `RColorBrewer` palette by specifying its name.
- `brewer.pal(n, name)` returns the hexadecimal color code of the palette.

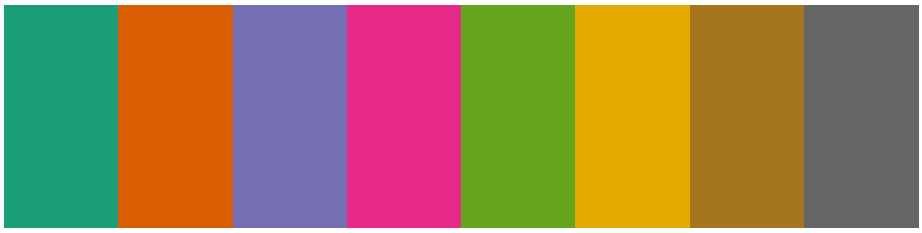
The two arguments:

`n` = Number of different colors in the palette, minimum 3, maximum depending on palette.

`name`= A palette name from the lists above. For example name = RdBu.

Example 1: Display the first 8 colors of palette `Dark2`.

```
# View a single RColorBrewer palette by specifying its name
display.brewer.pal(n = 8, name = 'Dark2')
```



Dark2 (qualitative)

Example 2: Return the hexadecimal of the first 8 colors of palette `Dark2`.

```
# Hexadecimal color specification
kable(t(brewer.pal(n = 8, name = "Dark2")))
```

#1B9E77	#D95F02	#7570B3	#E7298A	#66A61E	#E6AB02	#A6761D	#666666

A. Functions Calling Specific `rcolorbrewer` Palette in `ggplot()`

The following color scale functions are available in ggplot2 for using the `rcolorbrewer` palettes:

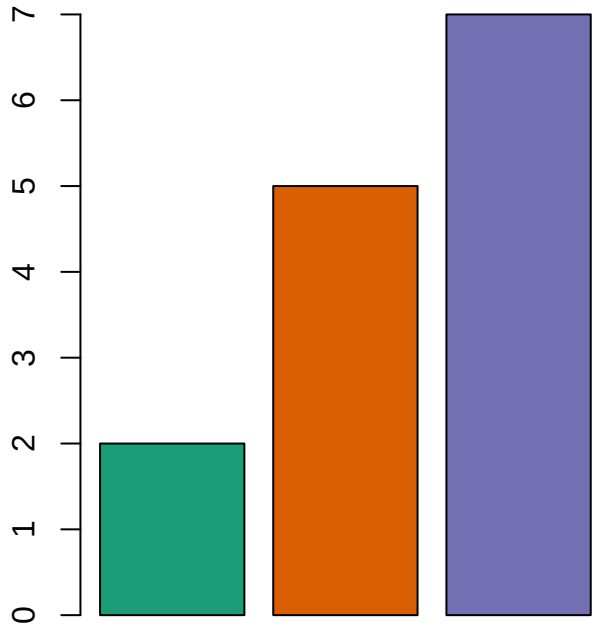
`scale_fill_brewer()` for box plot, bar plot, violin plot, dot plot, etc.

`scale_color_brewer()` for lines and points

B. Functions Calling Specific `rcolorbrewer` Palette in Base Plots

The function `brewer.pal()` is used to generate a vector of colors.

```
# Barplot using RColorBrewer
barplot(c(2,5,7), col = brewer.pal(n = 3, name = "Dark2"))
```


0
1
2
3
4
5
6
7